Masar Final project

Introduction

In this project, we developed a web application to study and test common web security vulnerabilities. Four vulnerabilities were implemented and tested in the application. After identifying the vulnerabilities, the application was modified to fix them and improve its security.

The vulnerabilities tested were:

- SQL Injection
- OS Command Injection
- Cross-Site Scripting (XSS)
- File Upload Vulnerability

Three machines running **Kali Linux** were used during the security testing process.

Students	Name
1	Faris Ishaq
2	Wadee Alsallq
3	Laith naser
4	Abdullah Altalafha

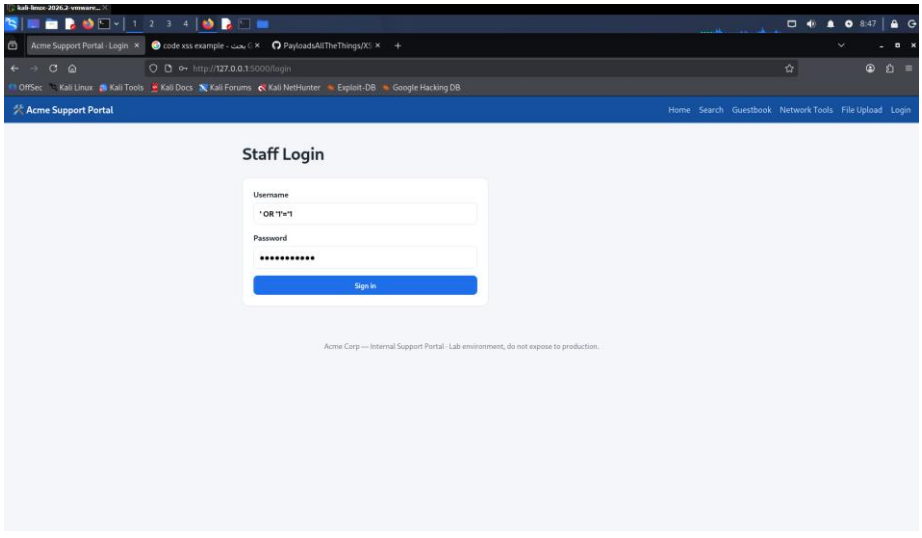
Vulnerabilitys Test

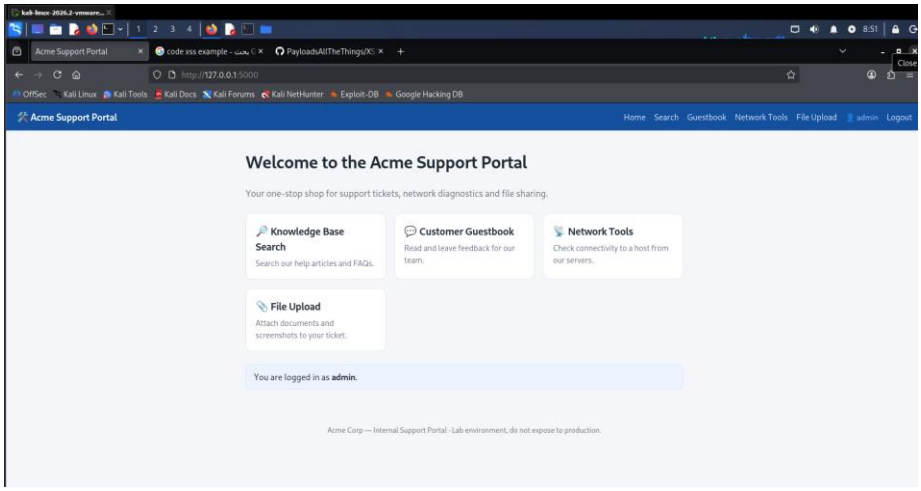
1. SQL Injection

The first vulnerability tested was **SQL Injection**. This vulnerability occurs when user input is handled insecurely within database queries.

Before Fixing the Vulnerability:

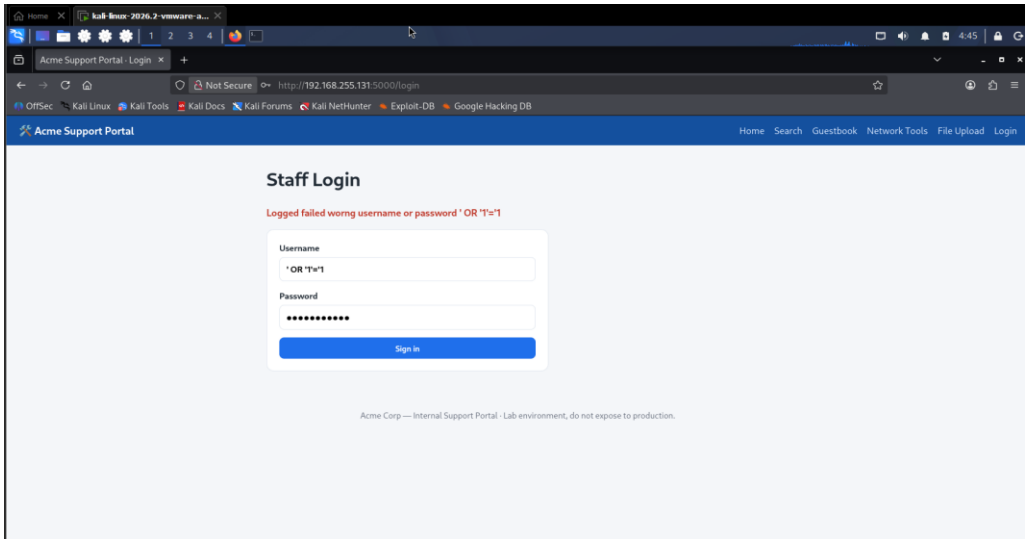






The vulnerability was tested in the lab environment to verify that it could be exploited

After Fixing the Vulnerability:



The SQL Injection vulnerability was successfully fixed by using parameterized queries and validating user input. The application was then retested to confirm that malicious SQL queries could no longer be executed

Fixing this vulnerability helps prevent unauthorized database access, data manipulation, and exposure of sensitive information

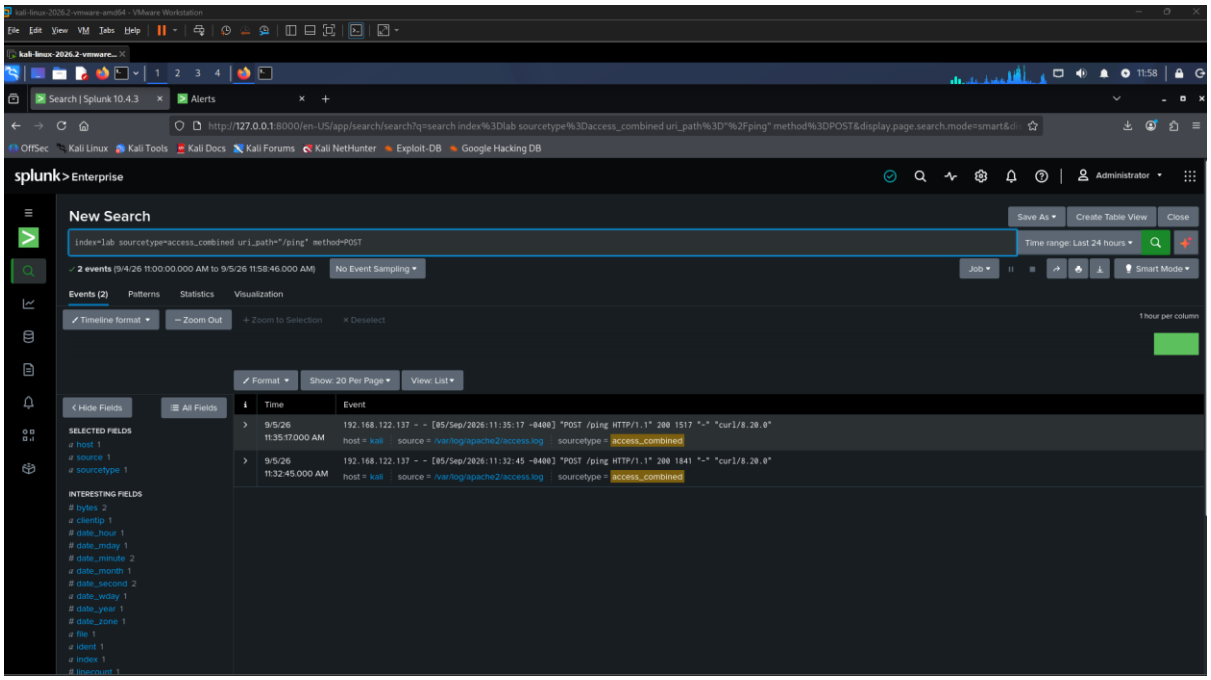
2. OS Command Injection

The second vulnerability tested was **OS Command Injection**. This vulnerability can occur when user input is passed directly to operating system commands without proper validation.

Before Fixing the Vulnerability:

The vulnerability was tested in the lab environment to verify that commands could be executed through the application.

Image:



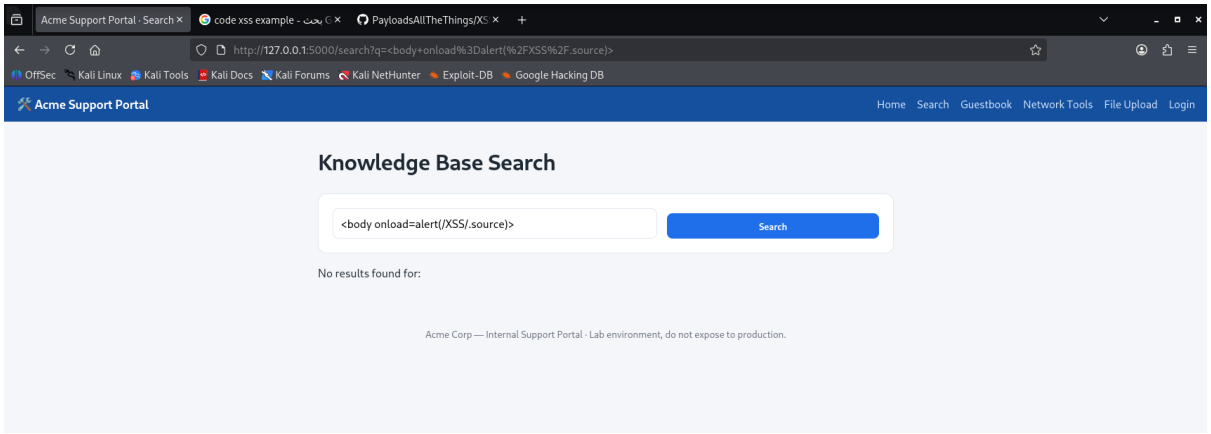
3. Cross-Site Scripting (XSS)

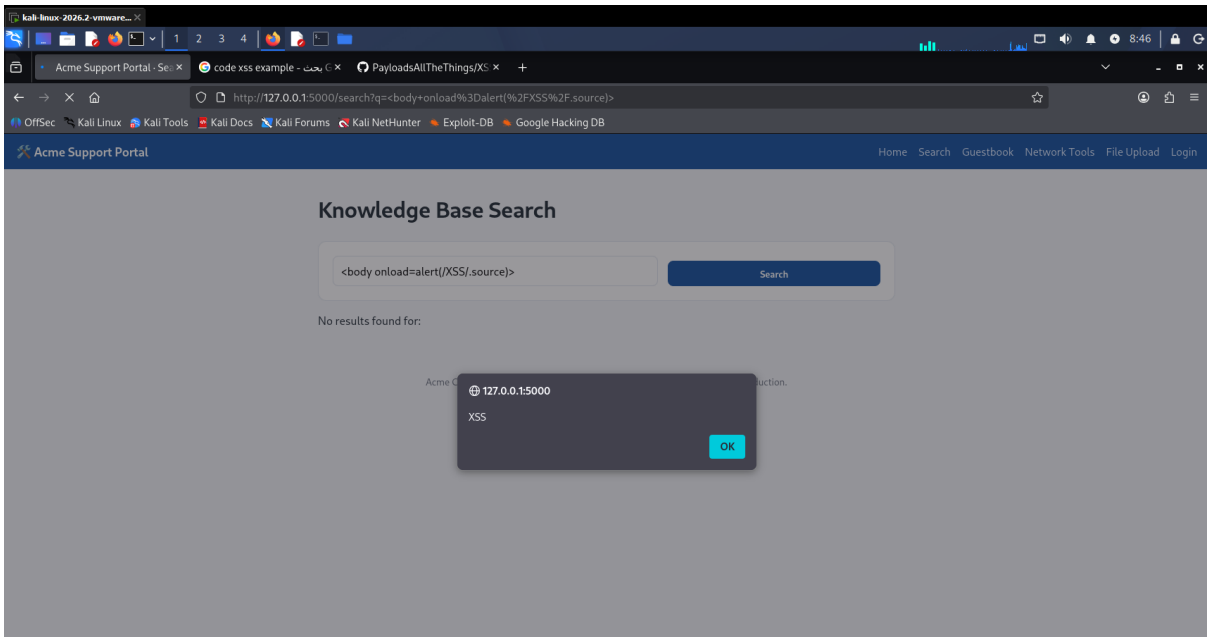
The third vulnerability tested was **Cross-Site Scripting (XSS)**. This vulnerability occurs when user input is displayed by the application without proper validation or encoding.

Before Fixing the Vulnerability:

The application was tested to verify that JavaScript content could be executed through a vulnerable input field.

Image:

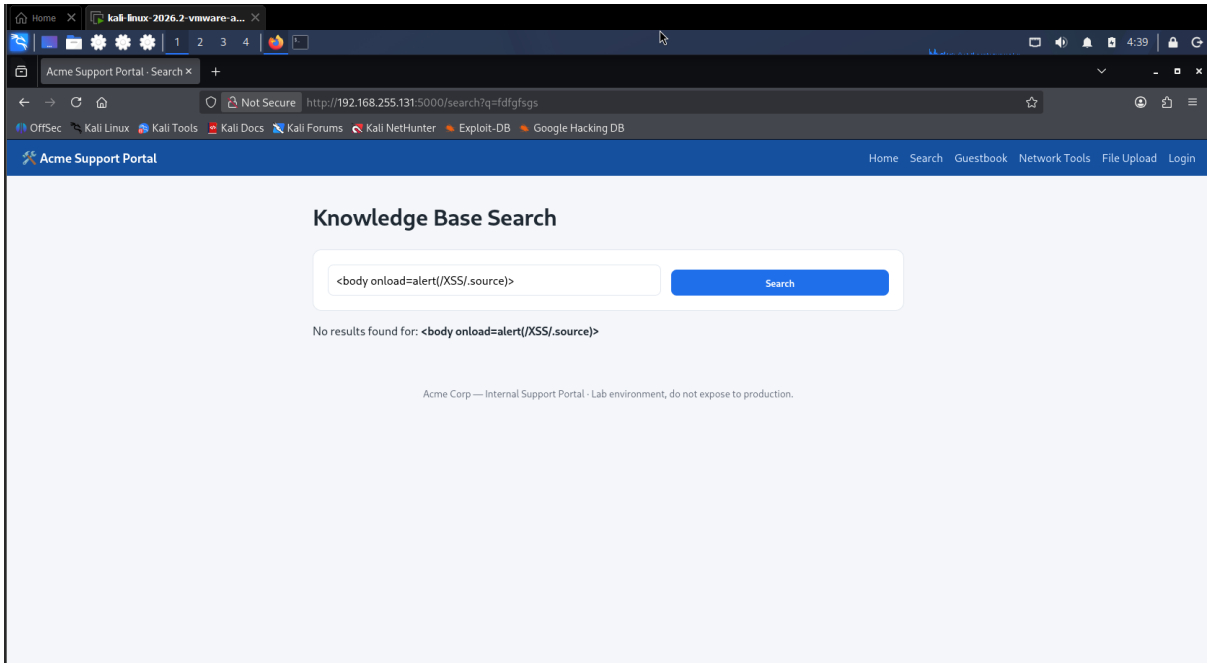




After Fixing the Vulnerability:

Input validation and output encoding were implemented to prevent unauthorized scripts from being executed.

Image:



XSS Vulnerability Remediation

The XSS vulnerability was successfully fixed by implementing proper input validation and output encoding. The application was then retested to confirm that malicious scripts could no longer be executed.

Fixing the XSS vulnerability protects users from malicious JavaScript execution, session and cookie theft, and unauthorized actions, improving the overall security of the application.

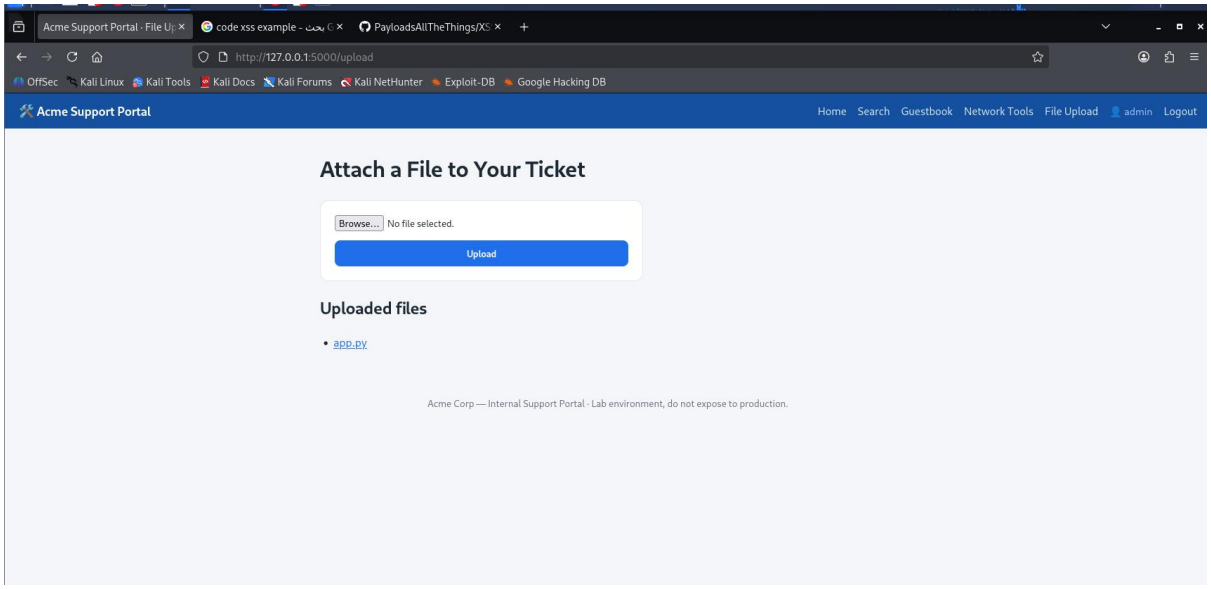
4. File Upload Vulnerability

The fourth vulnerability tested was the **File Upload Vulnerability**. This vulnerability occurs when an application does not properly validate uploaded files.

Before Fixing the Vulnerability:

The file upload functionality was tested to verify whether unauthorized file types could be uploaded.

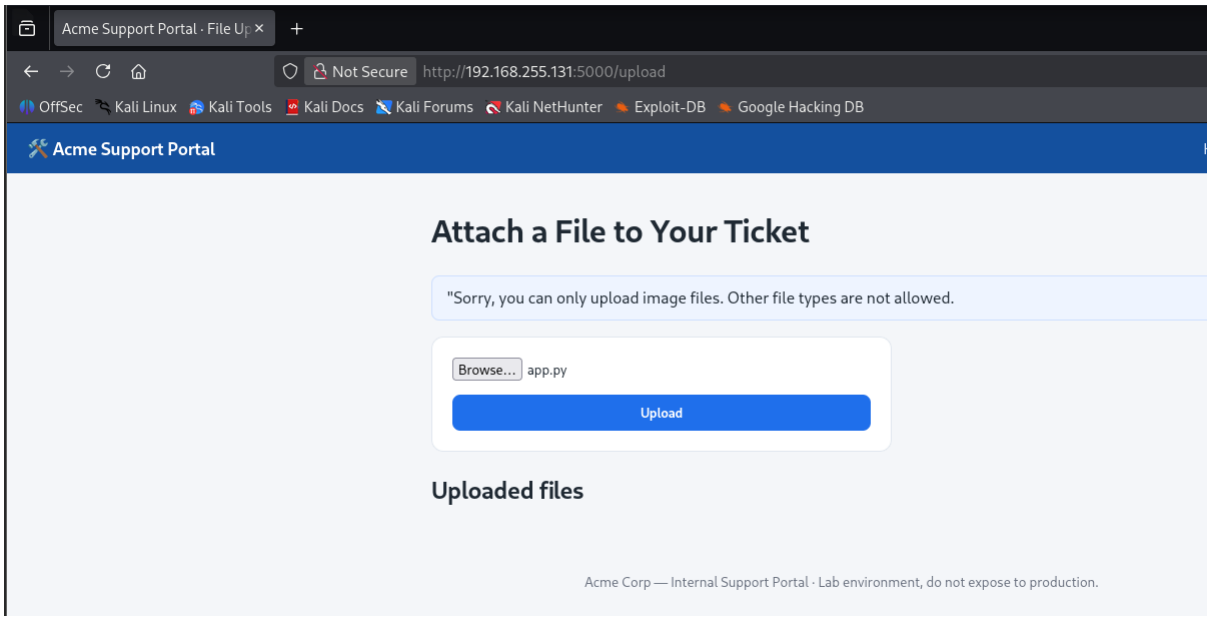
Image:



After Fixing the Vulnerability:

The file upload functionality was secured by applying appropriate file type, extension, and content validation.

Image:



File Upload Vulnerability Remediation

The File Upload vulnerability was successfully fixed by allowing the application to accept only image files. Other file types, such as Python files or executable files, are rejected and cannot be uploaded.

This fix helps prevent attackers from uploading malicious files, webshells, or other harmful content to the application

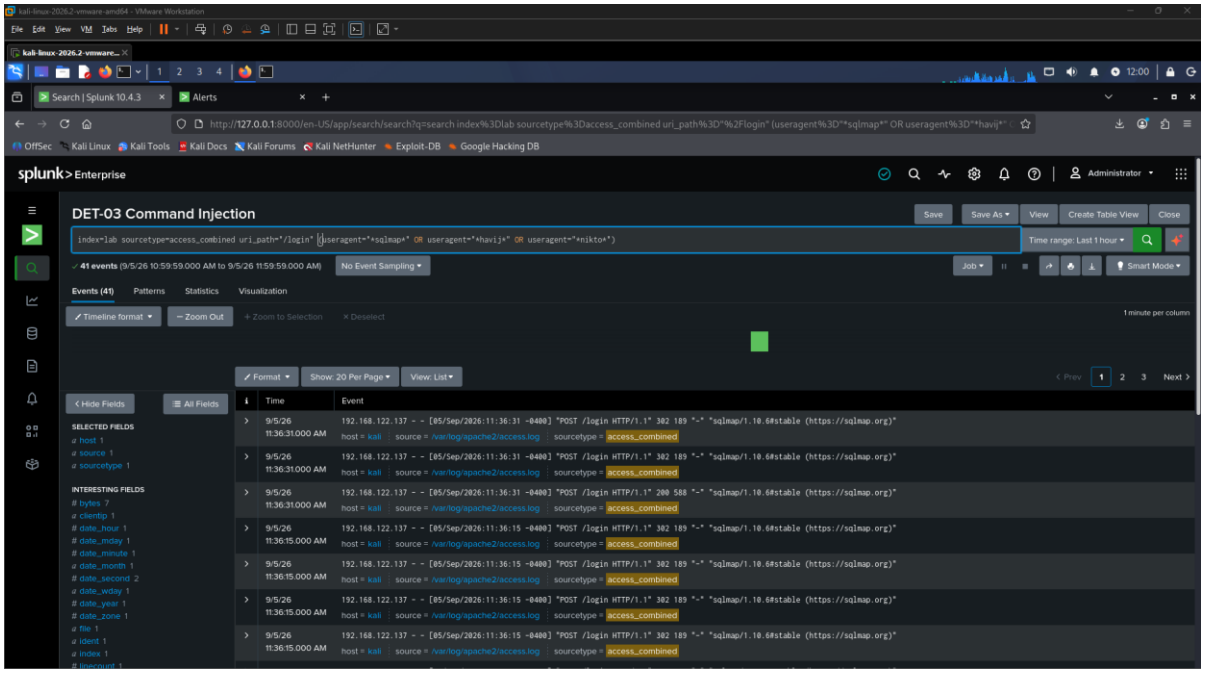
Defensive Team / Incident Response

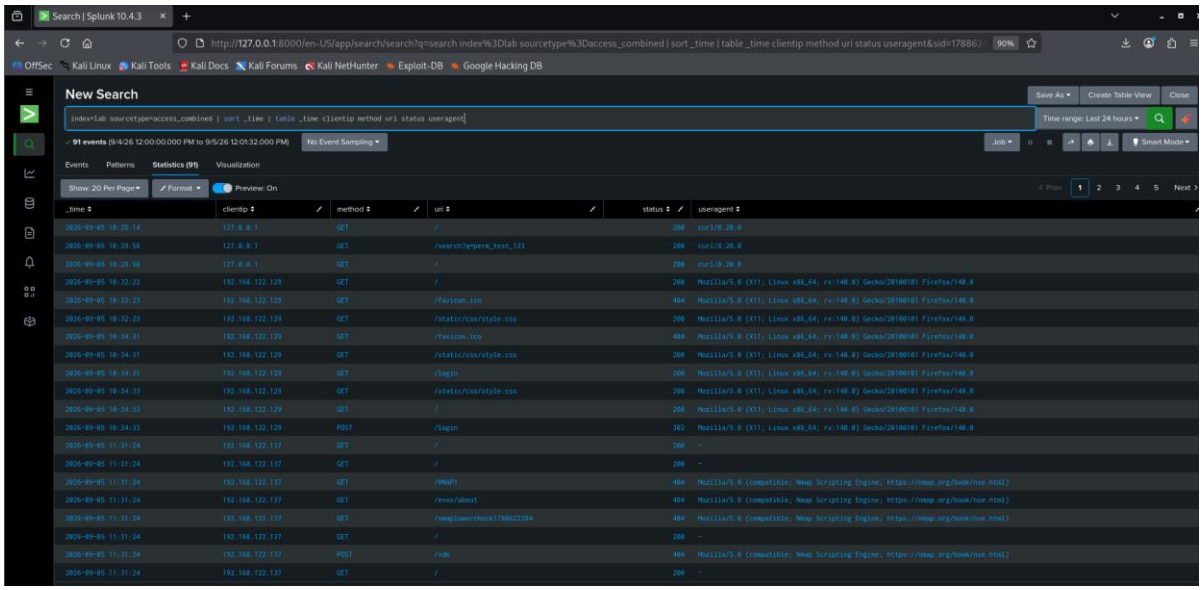
The attack timeline was reconstructed by analyzing and correlating Apache logs and Bash history to identify the attacker's requests, executed commands, and attack sequence.

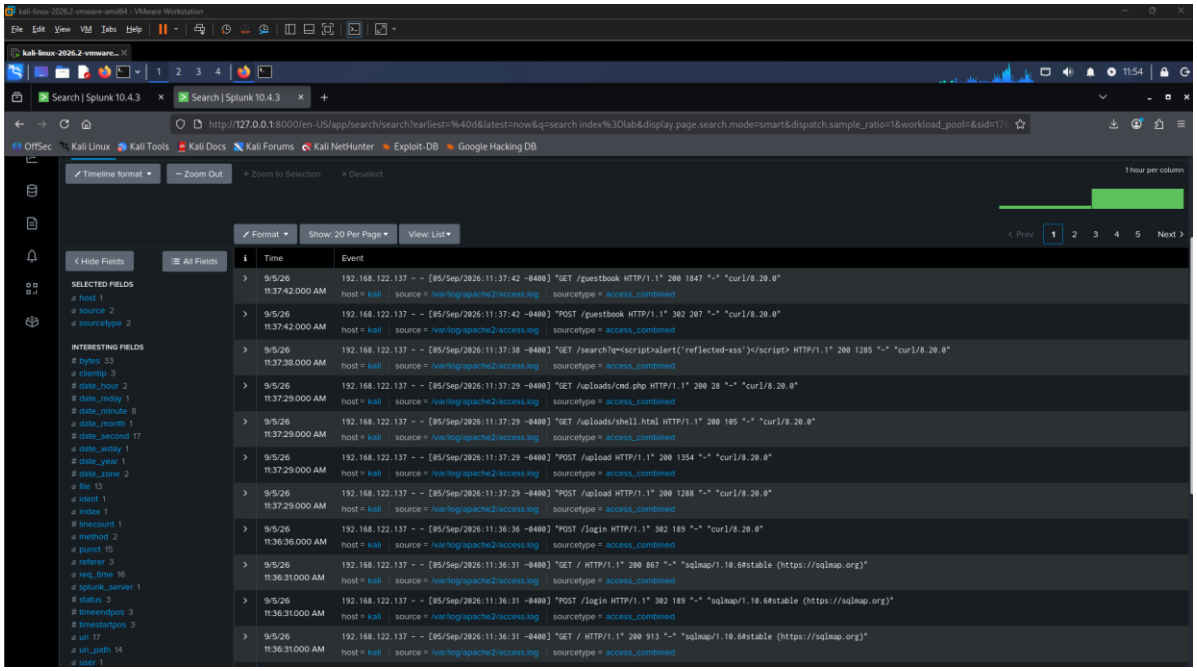
VM1 was then investigated to locate and remove the webshell and other attacker artifacts. The system was also checked to ensure that no persistence mechanisms remained.

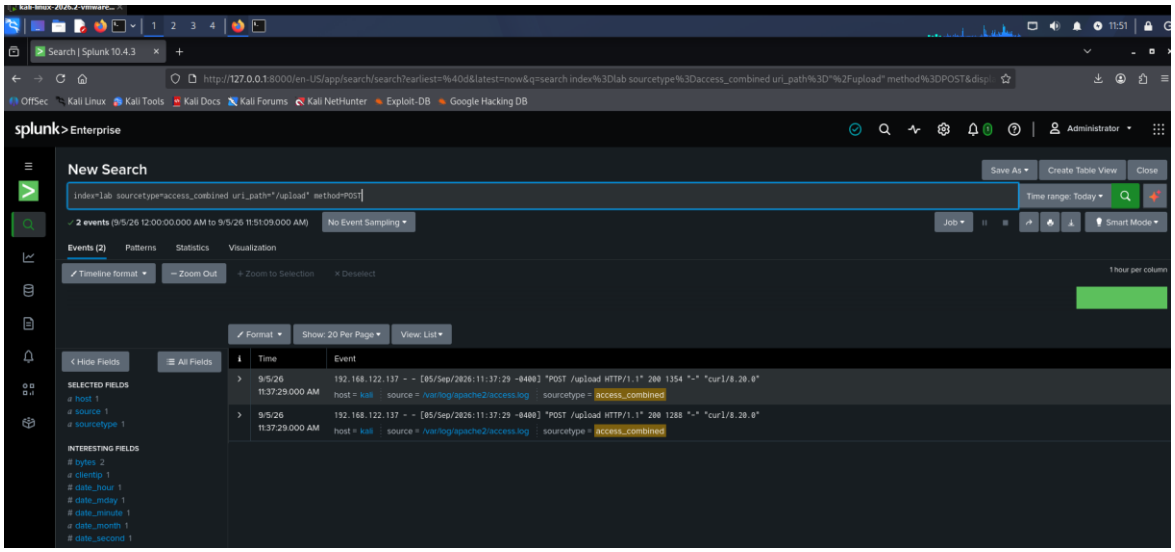
Detection queries were created for each identified vulnerability and tested in the SIEM. The queries were successfully validated again during Student 4's re-exploitation test, confirming that the attacks could still be detected.

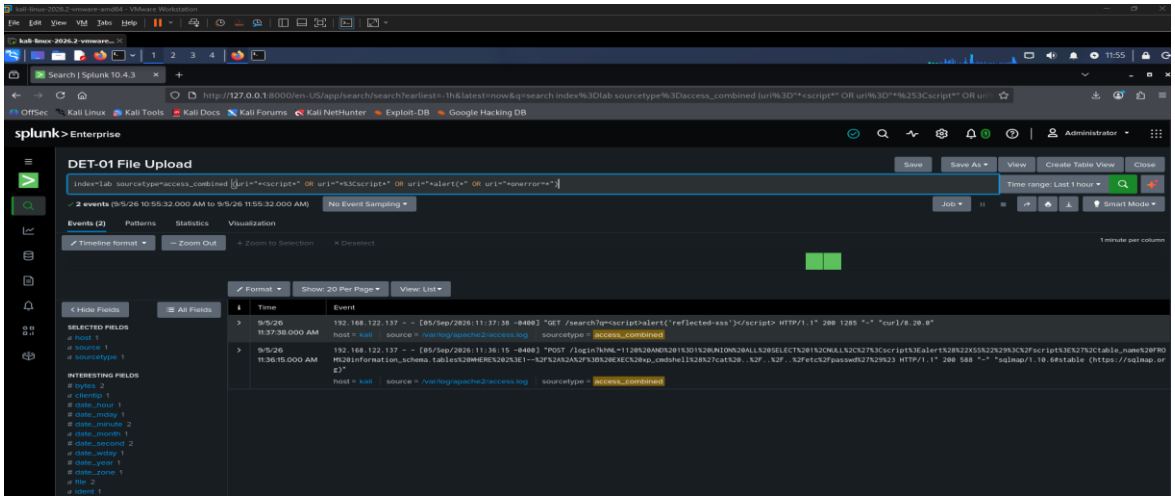
Images:


splunk> Enterprise
DET-03 Command Injection
index=lab sourcetype=access_combined uri_path="/login" useragent="*sqlmap* OR useragent="*havij*" OR useragent="*nikto*")
41 events (9/5/26 10:59:59.000 AM to 9/5/26 11:59:59.000 AM)
Events (41) Patterns Statistics Visualization
Time Event
9/5/26 11:36:31.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:31 -0400] "POST /login HTTP/1.1" 302 189 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"
9/5/26 11:36:31.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:31 -0400] "POST /login HTTP/1.1" 302 189 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"
9/5/26 11:36:31.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:31 -0400] "POST /login HTTP/1.1" 200 588 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"
9/5/26 11:36:15.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:15 -0400] "POST /login HTTP/1.1" 302 189 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"
9/5/26 11:36:15.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:15 -0400] "POST /login HTTP/1.1" 302 189 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"
9/5/26 11:36:15.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:15 -0400] "POST /login HTTP/1.1" 302 189 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"
9/5/26 11:36:15.000 AM 192.168.122.137 - - [05/Sep/2026:11:36:15 -0400] "POST /login HTTP/1.1" 302 189 "-" "sqlmap/1.10.6#stable (https://sqlmap.org)"


New Search
index=lab sourcetype=access_combined | sort _time | table _time clientip method uri status useragent
91 events (9/4/26 12:00:00.000 PM to 9/5/26 12:05:32.000 PM)
Events Patterns Statistics (91) Visualization
_time clientip method uri status useragent







Machines Used

Three machines were used during the project

Machine	Operating System	Primary Function	Observed Evidence
VM1 Target	Kali Linux	Vulnerable web app, Apache, log source	Acme Support Portal, upload/search/login endpoints
VM2 SIEM	Kali Linux	Splunk Enterprise, indexing, searches, alerts	Splunk Enterprise at local web interface
VM3 Attacker	Kali Linux	Black-box testing and exploitation	HTTP requests from attacker source and SQLMap activity

Conclusion

In this project, four common web application vulnerabilities were implemented and tested: **SQL Injection, OS Command Injection, Cross-Site Scripting (XSS), and File Upload Vulnerability**.

After identifying the vulnerabilities, appropriate security measures were implemented to fix them. The application was then tested again to verify that the vulnerabilities had been successfully addressed.